CS2201 Computer Organization & Architecture

L2: Review of Number Representation



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Number Systems

- Representation of positive numbers same in most systems
- Major differences are in how negative numbers are represented
- Two major schemes:
  - **Sign and magnitude**
  - **Twos complement**

Sign and Magnitude Representation

- Use fixed length binary representation
- Use left-most bit (called most significant bit or MSB) for sign:
  - 0 for positive
  - 1 for negative



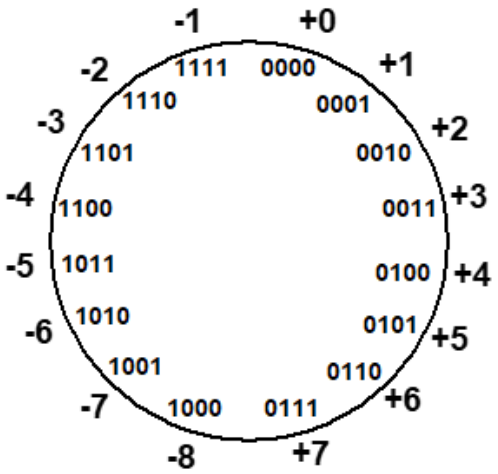
- Example: $+18_{ten} = 00010010_{two}$

  $-18_{ten} = 10010010_{two}$

Two's Complement Representation

- Use fixed-length representation, but no explicit sign bit:
  - 2's complement: To form a negative number, start with the given number, subtract one, and then complement each bit, or first complement each bit, and then add 1.

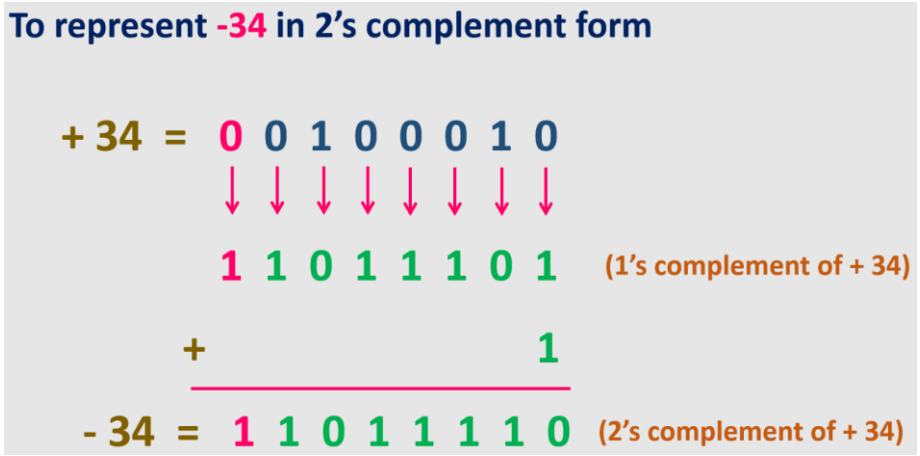
Twos Complement



- Only one representation for 0
- One more negative number than positive number

Decimal magnitude	Binary number	
	Positive	Negative
0	0000	
1	0001	1111
2	0010	1110
3	0011	1101
4	0100	1100
5	0101	1011
6	0110	1010
7	0111	1001
8		1000

2's Complement Numbers



- The two's complement range for an n-bit integer -2^{n-1} to $2^{n-1} - 1$
- 8-bit (1 byte): -128 to 127
- 16-bit (1 short): -32768 to 32767
- 32-bit (1 int): -2147483648 to 2147483647

Comparison of Representations

Sign-magnitude

000 = +0

001 = +1

010 = +2

011 = +3

100 = - 0

101 = - 1

110 = - 2

111 = - 3

2's complement

000 = +0

001 = +1

010 = +2

011 = +3

100 = - 4

101 = - 3

110 = - 2

111 = - 1

Addition and Subtraction of Numbers

Twos Complement Calculations

- **If carry-in to sign = carry-out then ignore carry**

4	0100	-4	1100
+ 3	0011	+ (-3)	1101
7	0111	-7	11001

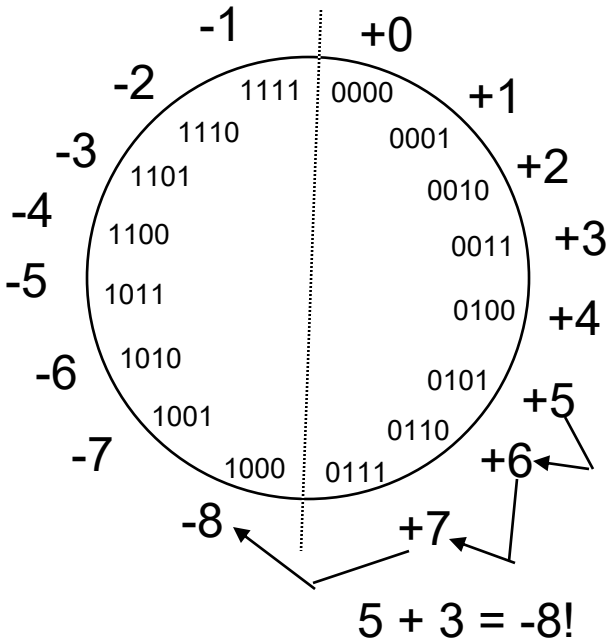
- **if carry-in differs from carry-out then overflow**

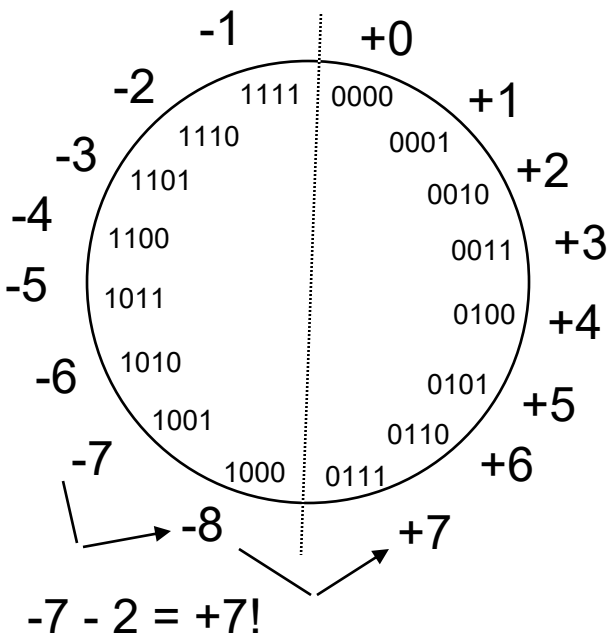
4	0100	-4	1100
- 3	1101	+ 3	0011
1	10001	-1	1111

Simpler addition scheme makes twos complement the most common choice for integer number systems within digital systems

Overflow Conditions

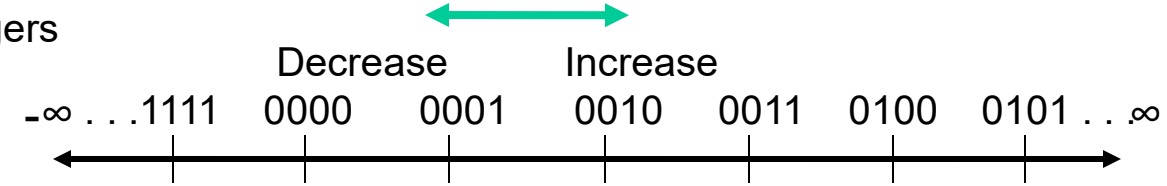
- Add two positive numbers to get a negative number
- Add two negative numbers to get a positive number

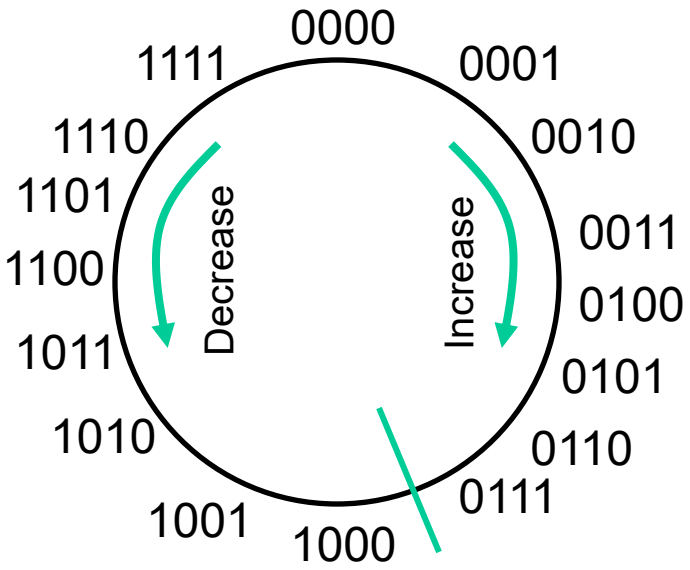




Overflow and Finite Universe

Infinite universe of integers
No overflow





Finite Universe of 4-bit Binary integers

Forbidden fence

Overflow Conditions

```
  5        0 1 1 1
           0 1 0 1
  3        0 0 1 1
 ---      ---------
 -8        1 0 0 0
```

Overflow

```
 -7        1 0 0 0
           1 0 0 1
 -2        1 1 1 0
 ---      ---------
  7      1 0 1 1 1
```

Overflow

```
  5        0 0 0 0
           0 1 0 1
  2        0 0 1 0
 ---      ---------
  7        0 1 1 1
```

No overflow

```
 -3        1 1 1 1
           1 1 0 1
 -5        1 0 1 1
 ---      ---------
 -8      1 1 0 0 0
```

No overflow

Overflow when carry in to sign (Cin) does not equal carry out (Cout)

Overflow Calculations

For the following operations on 5 bit 2-complement system, which one will have an overflow? Why/Why not?

- 10 + 12
- -6 + -8
- -13 - 5

Overflow Calculations

10 + 12

1000

10→ 01010

12→ 01100

\+ 10110

Cin ≠ Cout

OVERFLOW

-6 + -8

1000

-6→ 11010

-8→ 11000

110010 (-14)

Cin = Cout

NO OVERFLOW

-13 - 5 → -13 +2C {5}

0011

-13 → 10011

2C{5} → 11011

\+ 101110

Cin ≠ Cout

OVERFLOW

Positive Integers

- Decimal system: made of 10 digits, {0,1,2, . . . , 9}

$$41 = 4\times10^1 + 1\times10^0$$

$$255 = 2\times10^2 + 5\times10^1 + 5\times10^0$$

- Binary system: made of two digits, {0,1}

  Ex: what is the decimal equivalent of **00101001**?

$$= 0\times2^7 + 0\times2^6 + 1\times2^5 + 0\times2^4 + 1\times2^3 + 0\times2^2 + 0\times2^1 + 1\times2^0 = 32+8+1 = 41$$

- Hexadecimal System: the digits in base 16 are
  {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F}

Hexadecimal System

Denary/Decimal	Binary	Hexadecimal
Base 10 Number System	Base 2 Number System	Base 16 Number System
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

❖0x as a prefix is the convention for hexadecimal number: **Ex: 0x4B2A**

0x428 = ?

$= 4x16^2+2x16^1+ 8x16^0$

$= 1024+32+8 = 1064$

0xAD4 = ?

$= 10x16^2+13x16^1+ 4x16^0$

$= 2560+208+4 = 2772$

Converting Binary to Hexadecimal

- Mark groups of four (from right)
- Convert each group

10101011

10101011

A B

10101011 is AB in base 16 or **0xAB**

1010001101011110

1010001101011110

A 3 5 E

Hence, 1010001101011110 = 0xA35E

Denary/Decimal	Binary	Hexadecimal
Base 10 Number System	Base 2 Number System	Base 16 Number System
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/